Technical Design Monograph

Cross-Platform Research Development Assistant for Mathematical Finance, Macroeconomics, and Institutional Model Engineering

Prepared for Management Review

April 20, 2026

Contents

1 Executive Summary and Recommendation **1**
- 1.1 Executive Summary . . . 1
- 1.2 Recommendation . . . 2

2 Problem Definition **3**
- 2.1 The actual problem is not "paper search" or "AI coding" . . . 3
- 2.2 Why the existing tool stack is insufficient . . . 3
  - 2.2.1 Paper tools . . . 3
  - 2.2.2 Coding assistants . . . 4
  - 2.2.3 Manual notes . . . 4
- 2.3 Field-specific severity of the problem . . . 4

3 Design Goals and Non-Goals **5**
- 3.1 Primary design goals . . . 5
- 3.2 Non-goals for the initial phases . . . 5

4 System Overview **6**
- 4.1 One-sentence architectural summary . . . 6
- 4.2 Core architectural principle . . . 6
- 4.3 High-level layered architecture . . . 6
- 4.4 System context diagram . . . 7
- 4.5 Primary data flow . . . 7
- 4.6 Why this architecture is the right one . . . 7

5 Detailed Component Design **8**
- 5.1 Component inventory . . . 8
- 5.2 Proposed repository layout . . . 8
  - 5.2.1 File-by-file implementation intent . . . 10
- 5.3 Data ownership by component . . . 11

6 Data Model and Schemas **12**
- 6.1 Why explicit schemas are necessary . . . 12
- 6.2 Paper record schema . . . 12
- 6.3 Formal schema constraints . . . 13
- 6.4 Schema evolution policy . . . 14

6.5 Link record schema . . . 14
6.6 Audit record schema . . . 14
6.7 Concrete record examples . . . 14
6.7.1 Example paper record . . . 14
6.7.2 Example link record . . . 15
6.7.3 Example claim-support audit record . . . 15

7 Paper Ingestion and Metadata Resolution Pipeline 17
7.1 Ingestion goals . . . 17
7.2 Ingestion pipeline . . . 17
7.3 Ingestion pseudocode . . . 17
7.3.1 Ingestion pipeline responsibilities by stage . . . 18
7.4 Ingestion sequence diagram . . . 19
7.5 Failure handling in ingestion . . . 19
7.5.1 Metadata reconciliation pseudocode . . . 19
7.5.2 Deduplication pseudocode . . . 20
7.6 Metadata reconciliation . . . 20

8 Research Store and Retrieval Layer 21
8.1 Why file-first storage is preferred initially . . . 21
8.2 Retrieval operations . . . 21
8.3 Retrieval pseudocode . . . 21

9 Assistant Integration Strategy 23
9.1 Why multiple adapters are necessary . . . 23
9.2 Recommended access paths . . . 23
9.3 Illustrative MCP tool surface . . . 23
9.3.1 Proposed MCP tool contracts . . . 23
9.3.2 MCP implementation responsibilities . . . 24
9.4 CLI equivalents . . . 25
9.4.1 Illustrative CLI commands . . . 25
9.4.2 Why CLI parity matters . . . 25

10 Version 2 Architecture: Multi-Parser Reading and Citation-Graph Discovery 26
10.1 Why the architecture needs a Version 2 layer . . . 26
10.2 Multi-parser paper understanding subsystem . . . 26
10.2.1 Parser output schema . . . 27
10.2.2 Consensus output schema . . . 27
10.2.3 Voting is not naive majority voting . . . 28
10.3 Semantic Scholar / OpenAlex discovery subsystem . . . 28
10.3.1 Discovery commands . . . 28
10.4 Survey-generation workflow . . . 28
10.5 Why this is better than manually using Google Scholar . . . 28

11 Code-Aware Research Workflows **30**
11.1 Paper-to-code linking workflow . . . 30
11.2 Equation-to-code linking workflow . . . 30
11.3 Reviewer-facing synthesis workflow . . . 30

12 Engineering Realities and Platform Risk Assessment **31**
12.1 Why this section matters . . . 31
12.2 WSL2-specific issues . . . 31
12.3 Windows-specific issues . . . 31
12.4 Linux-specific issues . . . 32
12.5 Python environment fragmentation . . . 32
12.6 PDF and literature ingestion problems . . . 32
12.7 Assistant integration inconsistency . . . 33
12.8 Operational and maintenance risk . . . 33

13 Testing and Validation Strategy **34**
13.1 Why testing matters unusually strongly here . . . 34
13.2 Test categories . . . 34
13.2.1 Unit tests . . . 34
13.2.2 Integration tests . . . 34
13.2.3 Platform tests . . . 34
13.2.4 Failure-case tests . . . 34
13.2.5 Assistant adapter tests . . . 34
13.3 Representative fixture set . . . 34
13.4 Subsystem failure-mode appendix summary . . . 35
13.5 Platform test matrix . . . 36

14 Security, Privacy, and Governance **37**
14.1 Security posture . . . 37
14.2 Security and trust-boundary diagram . . . 37
14.3 Governance requirements . . . 37
14.4 Security design rules . . . 37

15 Delivery Plan and Operational Ownership **39**
15.1 Phase 1: local MVP . . . 39
15.2 Personal desktop POC chapter . . . 39
15.2.1 Why the personal POC is the right first build . . . 39
15.2.2 Functional scope of the personal POC . . . 40
15.2.3 POC-specific file layout . . . 40
15.2.4 POC command surface . . . 41
15.2.5 POC MCP surface for Claude Code . . . 41
15.2.6 POC success criteria . . . 41
15.2.7 Why this POC remains aligned with the larger architecture . . . 41
15.2.8 Suggested staffing model . . . 42

15.2.9 Operational ownership model 42
15.2.10 MVP sprint-by-sprint execution plan 42
15.3 Phase 2: assistant integration 43
15.4 Phase 3: review and audit workflows 43
15.5 Phase 4: group scaling 43
15.6 Ownership model 44

16 Anticipated Skeptical Questions and Preemptive Answers 45
16.1 Why not just use SciSpace? 45
16.2 Why not just use Claude Code or Cursor better? 45
16.3 Why should this be cross-platform? 45
16.4 Can this team realistically build it? 45
16.5 Why should management believe this is not vaporware? 46
16.6 How Claude Code could implement from this document 46
16.6.1 Suggested implementation appendix workflow 46

17 Decision Memo for Management Review 47
17.1 Decision requested 47
17.2 What approval enables 47
17.3 What approval does not yet commit the division to 47
17.4 Recommended decision 47

Chapter 1

Executive Summary and Recommendation

1.1 Executive Summary

This monograph proposes a shared research-development assistant for a division of approximately fifty economists, mathematical-finance researchers, and research engineers who build mathematically sophisticated, code-intensive systems for large institutional users such as central banks and financial institutions.

The proposal responds to a concrete workflow failure that is now common across modern research-engineering environments:

- paper tools such as SciSpace help with literature reading but are not code-aware;
- coding assistants such as Claude Code, Cursor, and Copilot help with implementation but lack durable structured research memory;
- manual copy-paste workflows create context loss, fragile documentation, and repeated trivial errors;
- there is no shared system that knows how papers, equations, code, tests, and reviewer-facing explanations fit together.

The proposed system is not an assistant replacement. It is a **shared research intelligence layer** that sits beneath all assistants and exposes consistent research context across them. Its core responsibilities are:

1. ingest and structure papers;
2. maintain literature-aware research memory;
3. connect literature, equations, code, tests, and technical documents;
4. expose this context to Claude Code, Cursor, and Copilot;
5. support reviewer-facing, auditable technical writing.

The architectural recommendation is explicit:

> **Build a shared backend and multiple assistant adapters, rather than a tool tied to any one AI assistant.**

The first implementation should be local-first, file-first, inspectable, and incremental. The system should begin as a structured paper store plus retrieval/query layer, then evolve toward

richer group-wide use only after it proves useful in real workflows.

1.2 Recommendation

The recommendation is to approve a phased build of a **Cross-Platform Research Development Assistant** with the following strategic properties:

- assistant-agnostic core architecture;
- local-first and inspectable data model;
- explicit support for paper-to-code traceability;
- rigorous reviewer-oriented documentation workflows;
- bounded MVP scope with strong immediate value;
- deliberate engineering simplicity in the first version.

The rest of this monograph justifies that recommendation in technical, operational, and organizational detail.

Chapter 2

Problem Definition

2.1 The actual problem is not "paper search" or "AI coding"

The division's workflow challenge is often misdescribed as a search or summarization problem. That framing is too weak. The actual problem is better stated as follows:

> How can a research-engineering organization maintain a shared, auditable, code-aware understanding of the literature, equations, implementation choices, failure modes, and documentation obligations behind mathematically sophisticated institutional software?

This is a different problem from simply:

- reading papers,
- writing code,
- or using a chat assistant.

It is a systems problem in research memory, traceability, assistant interoperability, and reviewer confidence.

2.2 Why the existing tool stack is insufficient

2.2.1 Paper tools

Tools such as SciSpace are useful for reading, annotation, and local Q&A about papers. But they do not deeply integrate with:

- repository structure,
- implementation status,
- test coverage,
- monograph chapters,
- production failure modes,
- or equation-to-code traceability.

2.2.2 Coding assistants

Claude Code, Cursor, and Copilot are useful for implementation assistance. But absent a structured research backend, they lack:

- durable project-level literature memory,
- validated citation support,
- paper-to-code linkage,
- reviewer-facing technical synthesis,
- and a shared state across assistants.

2.2.3 Manual notes

Private notes, copied summaries, and ad hoc documents do not scale to a team of fifty. They also do not provide a reliable machine-readable layer that assistants can query.

2.3 Field-specific severity of the problem

This is especially serious in mathematical finance, macroeconomics, DSGE modeling, and structural estimation because work in these fields combines:

- mathematical derivation,
- econometric assumptions,
- numerical methods,
- simulation and estimation,
- code implementation,
- and skeptical technical review.

A tool that ignores any one of those layers becomes much less useful.

Chapter 3

Design Goals and Non-Goals

3.1 Primary design goals

The system should:

1. preserve structured research memory;
2. connect papers, equations, code, tests, and documentation;
3. work across Claude Code, Cursor, and Copilot;
4. remain auditable by researchers and engineers;
5. support skeptical reviewer-facing output generation;
6. reduce repeated context reconstruction;
7. improve maintainability of mathematically sophisticated production code.

3.2 Non-goals for the initial phases

The first versions should *not* attempt to be:

- a full enterprise knowledge-management platform,
- a perfect paper-understanding engine,
- a general-purpose theorem prover,
- a replacement for all assistants,
- a multi-user web product from day one,
- or a system that requires heavy infrastructure before delivering value.

These non-goals are important because they constrain the initial scope and improve feasibility.

Chapter 4

System Overview

4.1 One-sentence architectural summary

The proposed system is a **shared research intelligence backend** with multiple thin assistant adapters.

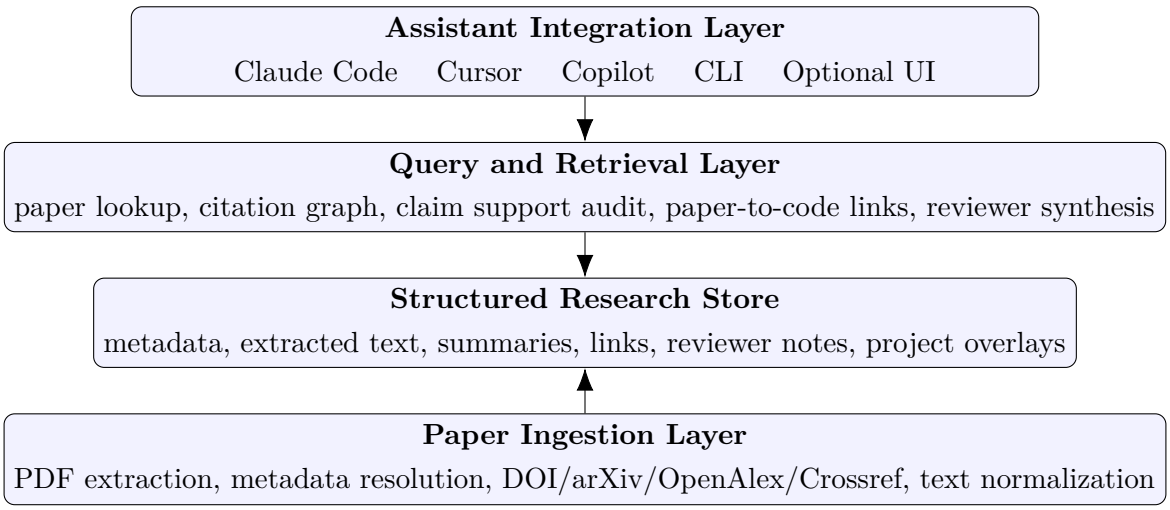
4.2 Core architectural principle

The most important architectural decision is the separation:

$$\text{shared research backend} + \text{assistant-specific adapters}$$

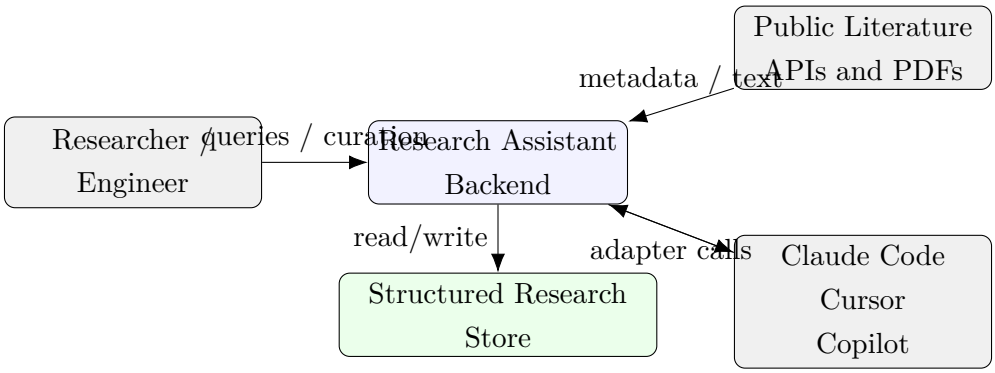
This is the only design that is robust to the division's multi-assistant environment.

4.3 High-level layered architecture



4.4 System context diagram

The following diagram shows the system boundary. The important point is that neither Claude Code, Cursor, nor Copilot owns the canonical research state. They query a shared backend. This is what makes the design usable across tools and robust to assistant changes.



4.5 Primary data flow

A typical request flows as follows:

1. An assistant or user asks a research-aware question.
2. The adapter converts the request into a backend query.
3. The backend retrieves structured paper summaries, code links, and metadata.
4. The backend returns structured context rather than free-form hidden state.
5. The assistant uses that context to write code, documentation, or an audit.

4.6 Why this architecture is the right one

This architecture is chosen because it directly addresses the main risks:

- **Assistant lock-in risk**: removed by separating backend from assistants.
- **Opaque state risk**: reduced by keeping the research store inspectable.
- **Cross-platform integration risk**: controlled by using multiple adapters.
- **Maintenance risk**: reduced by keeping the backend narrow and explicit.

Chapter 5

Detailed Component Design

5.1 Component inventory

The system should initially consist of the following components:

1. `paper_ingest`: import PDF, DOI, URL, arXiv.
2. `metadata_resolve`: query OpenAlex, Crossref, arXiv.
3. `text_extract`: PDF-to-text extraction and normalization.
4. `summary_builder`: generate structured summary records.
5. `link_manager`: maintain paper-to-code and paper-to-doc links.
6. `query_engine`: answer retrieval queries.
7. `audit_engine`: citation support and claim validation.
8. `assistant_adapters`: MCP, CLI, file-based, optional HTTP.

5.2 Proposed repository layout

The file layout below is intended to be specific enough that an engineer or Claude Code session could begin implementation directly from it. The design separates source code, research data, generated artifacts, and tests.

```
research-assistant/
  README.md
  pyproject.toml
  requirements.txt
  .gitignore
  src/
    research_assistant/
      __init__.py
      config.py
      paths.py
      errors.py
      logging.py
      ingest/
        __init__.py
        pdf_extract.py
```

```
      metadata_resolve.py
      normalize_text.py
      deduplicate.py
      source_manifest.py
    schemas/
      __init__.py
      paper_record.py
      link_record.py
      audit_record.py
      review_record.py
    storage/
      __init__.py
      file_store.py
      index_store.py
      cache_store.py
    summarize/
      __init__.py
      draft_summary.py
      claim_support.py
      reviewer_notes.py
      synthesis.py
    links/
      __init__.py
      paper_code_links.py
      paper_doc_links.py
      equation_links.py
    query/
      __init__.py
      paper_lookup.py
      citation_graph.py
      related_work.py
      search_index.py
    adapters/
      __init__.py
      mcp_server.py
      cli.py
      workspace_exports.py
      http_api.py
    tests/
      test_ingest.py
      test_metadata.py
      test_query.py
      test_claim_support.py
      test_paths.py
      test_links.py
      test_adapters.py
research/
  papers/
```

```
    raw/
    extracted/
  metadata/
  summaries/
  links/
  reviews/
  schemas/
  indices/
  caches/
docs/
  design/
  usage/
  schemas/
  runbooks/
```

5.2.1 File-by-file implementation intent

File or module	Intended responsibility
`config.py`	Global settings, environment resolution, path roots, optional API keys and local service configuration.
`paths.py`	Canonical path normalization across Linux, Windows, and WSL2.
`errors.py`	Typed exceptions for ingestion, metadata, adapter, and audit failures.
`pdf_extract.py`	PDF to text extraction, fallbacks, extraction status, and provenance capture.
`metadata_resolve.py`	OpenAlex / Crossref / arXiv resolution and metadata reconciliation.
`normalize_text.py`	Text cleanup, whitespace normalization, header/footer stripping, and chunking support.
`deduplicate.py`	Duplicate DOI/title/arXiv detection and canonical paper ID assignment.
`source_manifest.py`	Record the source origin, fetch date, and ingestion provenance of each paper.
`paper_record.py`	Validation and serialization for paper summary schema.
`link_record.py`	Validation and serialization for paper-to-code and paper-to-doc links.
`audit_record.py`	Validation and serialization for claim-support audit results.
`review_record.py`	Structured reviewer-facing synthesis and rebuttal notes.
`file_store.py`	Read/write operations for file-based research store.
`index_store.py`	Lightweight indices for search and lookup.

`cache_store.py`	Cache external metadata/API responses with provenance and expiry policy.
`draft_summary.py`	Generate first-pass structured summaries from extracted text and metadata.
`claim_support.py`	Determine whether a citation directly, indirectly, or weakly supports a claim.
`reviewer_notes.py`	Store reviewer objections, limitations, and caveats for a paper or method.
`synthesis.py`	Produce literature review drafts and structured synthesis across multiple papers.
`paper_code_links.py`	Create and update explicit mappings from papers to repository files and modules.
`paper_doc_links.py`	Map papers to LaTeX/Markdown documentation sections.
`equation_links.py`	Optional mapping from equations or method identifiers to code or docs.
`paper_lookup.py`	Query papers by title, DOI, arXiv, or semantic topic.
`citation_graph.py`	Retrieve citations, cited-by relationships, and neighboring literature.
`related_work.py`	Group papers by method family or implementation relevance.
`search_index.py`	Search over extracted text, summaries, and notes.
`mcp_server.py`	MCP tool definitions and request routing for Claude Code or compatible tools.
`cli.py`	Command-line entry points usable from any environment.
`workspace_exports.py`	Generate assistant-agnostic context files in the repo/workspace.
`http_api.py`	Optional local HTTP service for future integrations.

5.3 Data ownership by component

Component	**Writes**	**Reads**
paper_ingest	raw PDF, source manifest	URLs, DOI, arXiv IDs
metadata_resolve	metadata records	source manifest
text_extract	extracted text files	raw PDFs
summary_builder	summary drafts	metadata + extracted text
link_manager	paper/code/doc link files	summaries + repo paths
query_engine	query responses (ephemeral)	all structured stores
audit_engine	audit notes	summaries, citations, documents
assistant adapters	no canonical state by default	query layer and research store

Chapter 6

Data Model and Schemas

6.1 Why explicit schemas are necessary

A research system that relies only on free-form notes is difficult to query, validate, and maintain. Explicit schemas are therefore necessary to:

- reduce assistant hallucination,
- support consistent retrieval,
- allow manual inspection,
- and make the system auditable.

6.2 Paper record schema

A paper record should contain fields such as:

```
{
  "id": "string",
  "title": "string",
  "authors": ["string"],
  "year": 2026,
  "doi": "string|null",
  "arxiv_id": "string|null",
  "source_url": "string",
  "abstract": "string",
  "method_family": ["string"],
  "main_contribution": "string",
  "mathematical_core": "string",
  "assumptions": ["string"],
  "exactness_status": "direct|indirect|none|unknown",
  "invertibility_mechanism": "string|null",
  "jacobian_handling": "string|null",
  "known_defects": ["string"],
  "known_remedies": ["string"],
  "scientific_relevance": "string",
  "domain_relevance": "string",
  "implementation_implications": ["string"],
```

```
  "linked_code_files": ["string"],
  "linked_doc_sections": ["string"],
  "reviewer_objections": ["string"],
  "reviewer_responses": ["string"],
  "confidence_level": "high|medium|low",
  "curation_status": "draft|reviewed|approved"
}
```

6.3 Formal schema constraints

The first implementation should also define formal validation rules. A compact version of the core schema constraints is:

```
PaperRecord:
  required:
    - id
    - title
    - authors
    - year
    - source_url
    - main_contribution
    - confidence_level
    - curation_status
  constraints:
    id: nonempty string, unique
    title: nonempty string
    authors: nonempty list of strings
    year: integer in [1900, 2100]
    confidence_level: one of {high, medium, low}
    curation_status: one of {draft, reviewed, approved}

LinkRecord:
  required:
    - id
    - paper_id
    - target_type
    - target
    - relationship
  constraints:
    target_type: one of {code_file, document_section, equation, issue, test}

AuditRecord:
  required:
    - id
    - claim
    - cited_papers
    - support_classification
```

```
  constraints:
    support_classification:
      one of {direct_support, indirect_support, background_only,
             unsupported, insufficient_evidence}
```

6.4 Schema evolution policy

To keep the system stable for a team of fifty, schema evolution should be explicitly governed:

1. schema version numbers must be stored in records;
2. backward-incompatible changes require migration scripts;
3. adapters must reject unknown mandatory fields unless compatibility is explicitly declared;
4. changes to paper, link, or audit schemas should require review by both the engineer-owner and a domain reviewer.

6.5 Link record schema

A separate link record should represent many-to-many relationships such as:

- paper ↔ code file,
- paper ↔ document section,
- equation ↔ file/function,
- criticism ↔ mitigation implementation.

6.6 Audit record schema

An audit record should store:

- claim text,
- cited paper IDs,
- support classification,
- uncertainty notes,
- reviewer comments,
- timestamp and origin.

6.7 Concrete record examples

The following examples are intentionally concrete. They are included so that an implementation can begin from the document without inventing record formats later.

6.7.1 Example paper record

```
{
  "id": "paper_hoffman_2021_neutra",
  "title": "NeuTra-lizing Bad Geometry in Hamiltonian Monte Carlo Using Neural
      Transport",
```

```
  "authors": ["Matthew D. Hoffman", "Pavel Sountsov", "Joshua V. Dillon"],
  "year": 2021,
  "doi": null,
  "arxiv_id": "1903.03704",
  "source_url": "https://arxiv.org/abs/1903.03704",
  "method_family": ["HMC", "transport maps", "normalizing flows"],
  "main_contribution": "Uses an invertible neural transport as a preconditioner for
      exact HMC in transformed coordinates.",
  "mathematical_core": "Run HMC on U(T(z)) - log|det J_T(z)| rather than on U(q)
      directly.",
  "exactness_status": "exact after transformed-target HMC correction",
  "invertibility_mechanism": "inverse autoregressive flow",
  "jacobian_handling": "tractable log determinant from flow architecture",
  "known_defects": ["transport objective may not align with HMC efficiency", "tail
      behavior must be audited"],
  "known_remedies": ["shell diagnostics", "identity initialization", "bounded scale
      outputs"],
  "domain_relevance": "directly relevant to learned transport for posterior geometry",
  "confidence_level": "high",
  "curation_status": "reviewed"
}
```


6.7.2 Example link record


```
{
  "id": "link_neutra_transport_chapter",
  "paper_id": "paper_hoffman_2021_neutra",
  "target_type": "document_section",
  "target": "docs/chapters/ch16_transport_foundations.tex#sec:tf:exact_hmc",
  "relationship": "supports transformed-target HMC transport idea",
  "confidence_level": "high",
  "review_status": "reviewed"
}
```


6.7.3 Example claim-support audit record


```
{
  "id": "audit_transport_exactness_001",
  "claim": "An imperfect but invertible transport does not bias the final HMC chain if
       exact HMC is run on the transformed target including the Jacobian term.",
  "cited_papers": ["paper_hoffman_2021_neutra", "paper_papamakarios_2021_flows"],
  "support_classification": "direct_support",
  "evidence_summary": "NeuTra uses learned transport followed by HMC in warped space;
      normalizing-flow review supports exact change-of-variables density evaluation.",
  "limitations": ["Does not guarantee improved efficiency", "Requires exact Jacobian
      and stable gradients"],
  "reviewer_note": "Claim is about asymptotic correctness, not performance guarantee."
```

```
}
```

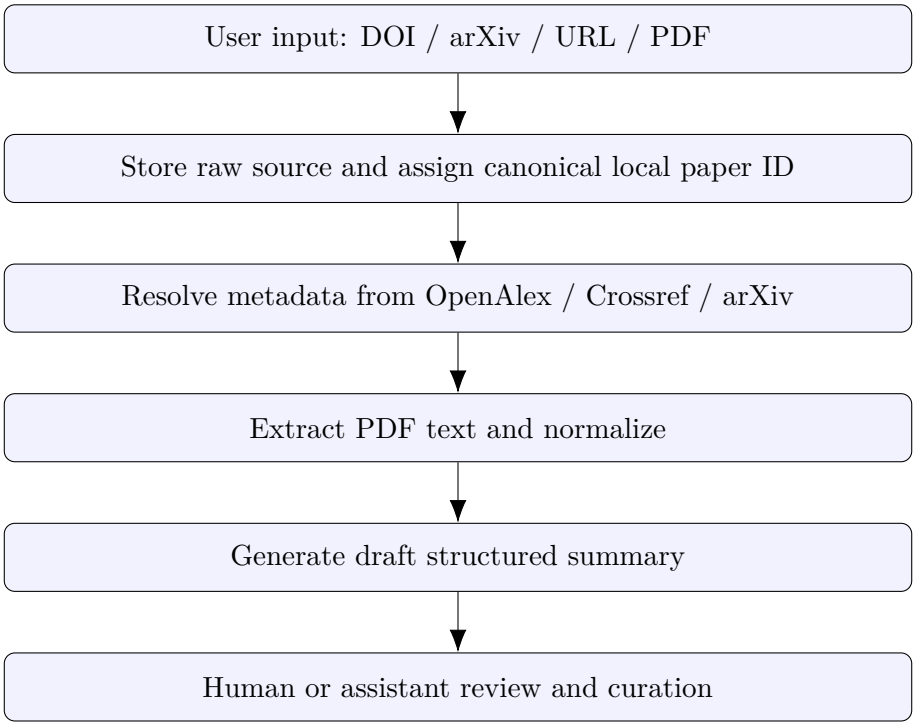
Chapter 7

Paper Ingestion and Metadata Resolution Pipeline

7.1 Ingestion goals

The ingestion layer should turn an external reference into a reproducible local research artifact.

7.2 Ingestion pipeline



7.3 Ingestion pseudocode

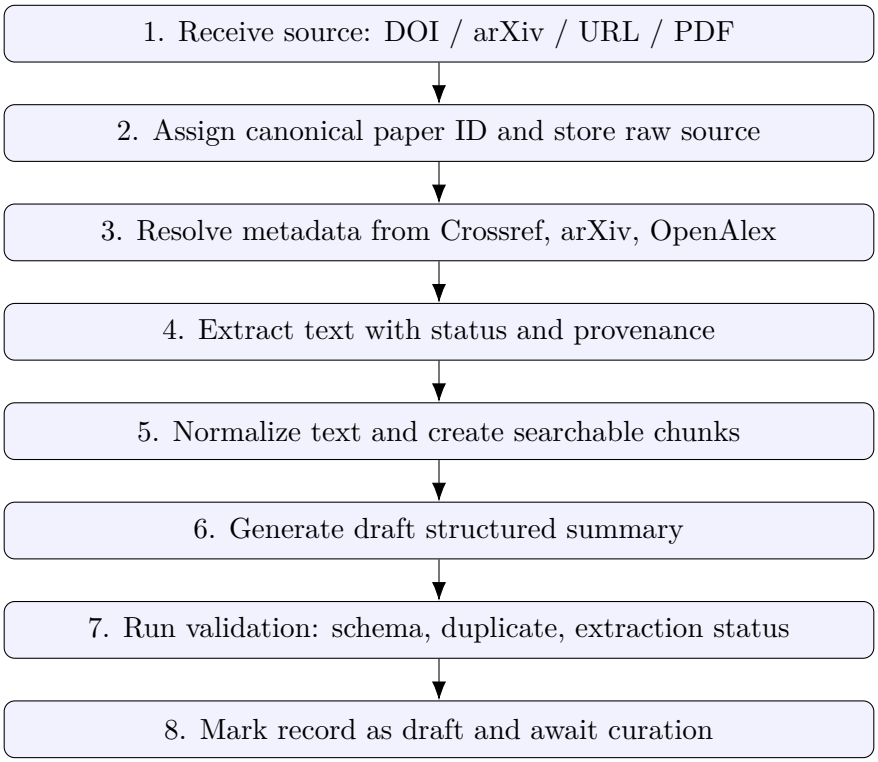
```
def ingest_paper(source):
    paper_id = assign_canonical_id(source)
    raw_path = store_raw_source(source, paper_id)
```

```
    metadata = resolve_metadata(source)
    extracted_text = extract_text(raw_path)
    normalized_text = normalize_text(extracted_text)

    summary = build_draft_summary(
        paper_id=paper_id,
        metadata=metadata,
        text=normalized_text,
    )

    write_metadata(paper_id, metadata)
    write_extracted_text(paper_id, normalized_text)
    write_summary_draft(paper_id, summary)

    return paper_id
```

7.3.1 Ingestion pipeline responsibilities by stage

1. **Canonical ID assignment.** The system assigns a stable internal paper ID before any downstream processing. This avoids path drift when metadata later changes.
2. **Raw source preservation.** The original PDF or fetched representation is stored unchanged for reproducibility.
3. **Metadata resolution.** DOI, arXiv, title, authors, year, and source provenance are retrieved from multiple services and reconciled.
4. **Text extraction and normalization.** Extracted text is normalized but never treated as a replacement for the raw source.
5. **Draft summary creation.** A first-pass structured summary is generated and marked as draft, not authoritative.
6. **Curation handoff.** The paper remains in draft state until reviewed by a human or a validated assistant workflow.

7.4 Ingestion sequence diagram



7.5 Failure handling in ingestion

The ingestion pipeline must never silently pretend success. Each stage should return an explicit status:

- `ok`,
- `partial`,
- `failed`,
- `requires_manual_review`.

A paper with failed PDF extraction may still have useful metadata, but it should not be treated as fully ingested. This distinction is important for reviewer trust.

7.5.1 Metadata reconciliation pseudocode

```
def resolve_metadata(source):
    candidates = []

    if has_doi(source):
        candidates.append(fetch_crossref(source.doi))
    if has_arxiv_id(source):
        candidates.append(fetch_arxiv(source.arxiv_id))

    candidates.append(fetch_openalex(source))
```

```
    merged = merge_metadata_candidates(candidates)
    validate_required_fields(merged)
    return merged
```

7.5.2 Deduplication pseudocode

```
def assign_canonical_id(source):
    matches = []
    matches += lookup_by_doi(source)
    matches += lookup_by_arxiv(source)
    matches += lookup_by_normalized_title(source)

    if len(matches) == 1:
        return matches[0].paper_id
    if len(matches) > 1:
        return resolve_duplicate_conflict(matches)
    return mint_new_paper_id(source)
```

7.6 Metadata reconciliation

A single source should not be treated as authoritative in all cases.

Recommended preference ordering:

1. DOI/Crossref for publication metadata,
2. arXiv for preprint metadata,
3. OpenAlex for citation graph and broad work relationships,
4. local human curation for final correctness if sources conflict.

Chapter 8

Research Store and Retrieval Layer

8.1 Why file-first storage is preferred initially

A file-first research store is preferred because it is:

- transparent,
- easy to version-control,
- robust to tool changes,
- debuggable by researchers,
- and suitable for local-first development.

8.2 Retrieval operations

The query layer should support operations such as:

- find paper by title/DOI/arXiv/topic,
- list citations and citing works,
- find papers related to a method family,
- read structured summary,
- find papers linked to code files,
- generate synthesis across a set of papers,
- audit support for a claim.

8.3 Retrieval pseudocode

```
def find_related_papers(topic, filters=None):
    candidate_ids = search_index(topic, filters)
    papers = load_paper_records(candidate_ids)
    ranked = rank_by_relevance(papers, topic)
    return ranked


def claim_support_audit(claim, cited_paper_ids):
    papers = [load_paper_record(pid) for pid in cited_paper_ids]
    evidence = extract_relevant_support(papers, claim)
```

```
    classification = classify_support(evidence, claim)
    return {
        "claim": claim,
        "papers": cited_paper_ids,
        "evidence": evidence,
        "classification": classification,
    }
```

Chapter 9

Assistant Integration Strategy

9.1 Why multiple adapters are necessary

Claude Code, Cursor, and Copilot differ significantly in:

- tool invocation capability,
- persistence model,
- editor integration,
- and how much structured context they can consume.

Therefore, the backend must expose more than one interface.

9.2 Recommended access paths

1. **MCP server** for Claude Code and any compatible environment.
2. **CLI tools** for universal availability.
3. **Workspace-generated files** for visibility inside all editors.
4. **Optional HTTP API** for future richer integrations.

9.3 Illustrative MCP tool surface

Possible MCP tools:

```
find_paper(query)
get_paper_summary(paper_id)
papers_citing(paper_id)
papers_cited_by(paper_id)
related_papers(topic)
paper_code_links(paper_id)
claim_support_audit(claim, paper_ids)
reviewer_synthesis(topic, paper_ids)
```

9.3.1 Proposed MCP tool contracts

To reduce ambiguity, the first implementation should define a stable tool contract. An illustrative version is:

```
find_paper(
  query: string,
  filters?: {
    year_min?: int,
    year_max?: int,
    method_family?: [string],
    curated_only?: bool
  }
) -> [
  {
    paper_id: string,
    title: string,
    year: int,
    relevance_score: float,
    curation_status: string
  }
]

get_paper_summary(
  paper_id: string
) -> {
  metadata: object,
  summary: object,
  links: [object]
}

claim_support_audit(
  claim: string,
  paper_ids: [string]
) -> {
  classification: string,
  evidence: [string],
  limitations: [string],
  confidence: string
}
```

9.3.2 MCP implementation responsibilities

The MCP adapter should be thin. It should:

- validate inputs,
- call the backend query functions,
- normalize outputs into assistant-friendly structures,
- avoid embedding business logic that belongs in the backend.

9.4 CLI equivalents

The same operations should also exist as CLI commands so the backend remains useful outside any specific assistant.

9.4.1 Illustrative CLI commands

```
ra paper find --query "neural transport hmc"
ra paper show --paper-id paper_hoffman_2021_neutra
ra paper citing --paper-id paper_hoffman_2021_neutra
ra audit claim --claim-file claim.txt --papers p1 p2 p3
ra links add --paper-id p1 --target src/foo.py --relationship "implements"
ra review synthesize --topic "transport maps for posterior geometry"
```

9.4.2 Why CLI parity matters

CLI parity is essential because it ensures the backend remains usable even in assistant environments that do not support rich tool orchestration. It also supports scripting, CI validation, and debugging outside IDEs.

Chapter 10

Version 2 Architecture: Multi-Parser Reading and Citation-Graph Discovery

10.1 Why the architecture needs a Version 2 layer

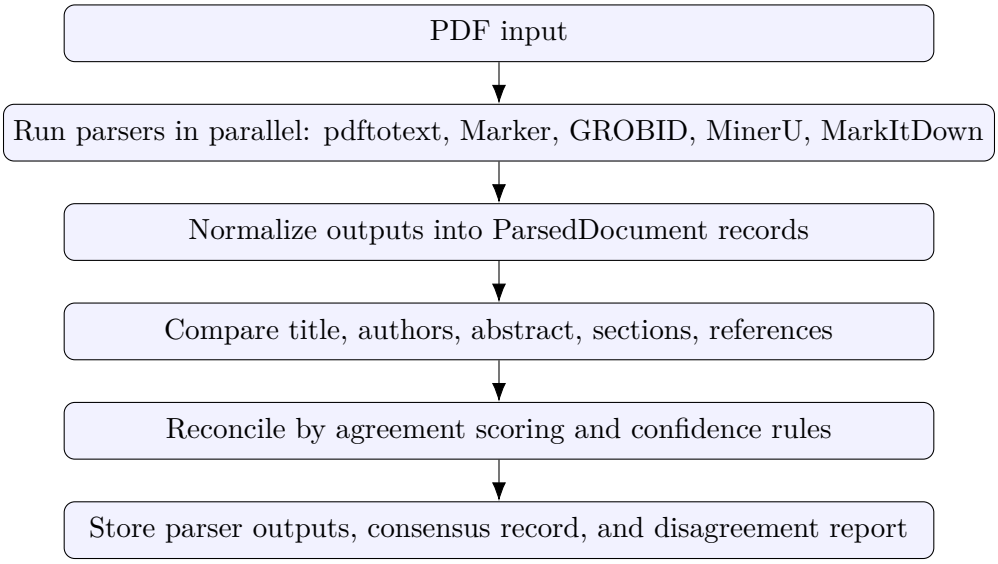
The personal POC revealed an important limitation: metadata APIs and simple PDF text extraction are not enough for robust local paper ingestion. A Google Scholar-like experience requires two separate capabilities:

1. **strong document understanding** for papers already in the local library;
2. **fast citation-graph discovery** for finding relevant papers beyond the local library.

These are different subsystems and should not be conflated. Local parsing answers "what does this paper say?" Citation graph discovery answers "which papers should I read next?"

10.2 Multi-parser paper understanding subsystem

No single open-source PDF parser is expected to be foolproof. Therefore the recommended design is a multi-parser consensus pipeline:



10.2.1 Parser output schema

Each parser should return the same abstract object:

```
ParsedDocument:
  parser_name: string
  parser_version: string
  title_candidates: [string]
  authors: [string]
  abstract: string
  body_text: string
  body_markdown: string
  section_headings: [string]
  references: [string]
  diagnostics: object
  parse_status: ok|partial|failed
```

10.2.2 Consensus output schema

The reconciler should produce:

```
ReconciledDocument:
  consensus_title: string|null
  consensus_authors: [string]
  consensus_abstract: string|null
  parser_agreement: object
  disagreements: [string]
  parse_confidence: high|medium|low
  requires_manual_review: bool
```

10.2.3 Voting is not naive majority voting

The system should use evidence-weighted reconciliation rather than raw majority voting. For example:

- title strings should be normalized and clustered by similarity;
- author lists should be compared by name overlap;
- abstracts should be checked for length and location sanity;
- parsers with corrupt or extremely short output should be penalized;
- disagreement should be preserved rather than hidden.

10.3 Semantic Scholar / OpenAlex discovery subsystem

The discovery subsystem should use external scholarly indices for breadth. The recommended order is:

1. Semantic Scholar as the primary citation graph provider;
2. OpenAlex as fallback and enrichment;
3. Crossref and arXiv for metadata validation;
4. optional later integration with Scite or Lens if institutional access justifies it.

10.3.1 Discovery commands

The CLI and MCP surfaces should eventually support:

```
ra discover --query "neural transport hmc posterior geometry"
ra citing --paper-id paper_hoffman_2021_neutra --depth 1
ra citing --paper-id paper_hoffman_2021_neutra --depth 2
ra references --paper-id paper_hoffman_2021_neutra
ra cluster-literature --seed paper_hoffman_2021_neutra --depth 2
ra survey-draft --topic "defects and remedies of NeuTra-style transport"
```

10.4 Survey-generation workflow

The intended literature-survey workflow is:

1. start from a seed paper or topic;
2. traverse citation graph to depth 1 or 2;
3. rank papers by relevance, citation relationship, recency, and local availability;
4. ingest or parse selected papers;
5. summarize and classify them;
6. draft a survey organized by method family, defects, remedies, and applicability;
7. keep human review as the final authority.

10.5 Why this is better than manually using Google Scholar

Google Scholar is excellent for discovery, but it is not code-aware and is not easily automatable. The proposed system does not attempt to replace Google Scholar globally. Instead, it combines:

- Semantic Scholar / OpenAlex for citation graph traversal;
- local parsers for deep paper reading;
- structured research memory for curation;
- Claude Code / CLI integration for code-aware documentation.

This is the architecture most likely to help researchers read and triage hundreds of papers while still connecting the output to implementation and technical documentation.

Chapter 11

Code-Aware Research Workflows

11.1 Paper-to-code linking workflow

1. researcher identifies a method paper,
2. system retrieves or creates paper record,
3. assistant or user proposes candidate code files,
4. explicit links are stored,
5. documentation can later cite both the paper and the implementation location.

11.2 Equation-to-code linking workflow

The system should support links of the form:

- equation label → monograph section,
- monograph section → code file/function,
- code file/function → paper or method family.

This is critical for skeptical review and maintainability.

11.3 Reviewer-facing synthesis workflow

1. user asks for a literature/design section,
2. system retrieves curated summaries,
3. assistant generates draft synthesis,
4. claim-support audit is run,
5. researcher reviews and finalizes.

Chapter 12

Engineering Realities and Platform Risk Assessment

12.1 Why this section matters

For a research-engineering team, engineering detail is not secondary. It is one of the main determinants of whether a good idea becomes a reliable tool or an expensive maintenance burden. This section therefore surfaces the platform and systems risks explicitly rather than minimizing them.

12.2 WSL2-specific issues

Potential issues:

- path translation between Windows host and Linux guest,
- slow file watching on mounted Windows filesystems,
- inconsistent permission behavior,
- intermittent DNS/network issues,
- tool behavior differences between files under '/home/...' and '/mnt/c/...',
- IDE host/guest integration quirks.

Mitigation:

- prefer storing active research stores under Linux-native paths,
- normalize all paths internally,
- avoid watcher-heavy architecture in the MVP,
- support explicit re-index commands,
- test WSL2 explicitly as a first-class environment.

12.3 Windows-specific issues

Potential issues:

- path separator differences,
- quoting and shell behavior differences,
- PowerShell vs bash command semantics,

- long path and Unicode path issues,
- local dependency install inconsistency.

Mitigation:

- use Python path abstractions internally,
- avoid shell-specific logic in the core backend,
- provide CLI wrappers that are shell-agnostic where possible,
- document supported environment assumptions clearly.

12.4 Linux-specific issues

Potential issues:

- distro-level package differences,
- missing system utilities,
- user-level installation drift across machines.

Mitigation:

- keep core dependencies minimal,
- prefer Python-based implementation where practical,
- document a reproducible setup process.

12.5 Python environment fragmentation

Potential issues:

- conda vs system python vs venv,
- package-version conflicts,
- institution-specific proxy/network restrictions,
- dependency drift between users.

Mitigation:

- pin dependencies,
- provide one supported environment recipe,
- isolate the tool in its own environment,
- keep optional dependencies clearly separated.

12.6 PDF and literature ingestion problems

Potential issues:

- malformed PDFs,
- scanned PDFs requiring OCR,
- inconsistent metadata across sources,
- duplicate paper resolution,
- incomplete citation graph coverage,
- brittle Google Scholar behavior.

Mitigation:

- keep raw PDF, extracted text, and curated summaries separate,
- use multiple metadata sources,
- prefer OpenAlex/Crossref/arXiv over brittle scraping,
- mark uncertainty explicitly.

12.7 Assistant integration inconsistency

Potential issues:

- Claude Code has richer tool integration than Copilot,
- Cursor may differ across versions/configurations,
- not every assistant will support the same retrieval path.

Mitigation:

- backend must be independent,
- MCP is only one access path, not the only path,
- CLI and workspace files serve as universal fallback.

12.8 Operational and maintenance risk

Potential issues:

- schema drift,
- stale summaries,
- broken links to code after refactors,
- low team adoption if workflow is too heavy,
- unclear ownership.

Mitigation:

- explicit schemas,
- curation status fields,
- periodic stale-link audits,
- low-friction MVP,
- explicit operational ownership.

Chapter 13

Testing and Validation Strategy

13.1 Why testing matters unusually strongly here

This system sits between literature, code, and documentation. Silent errors can therefore propagate into reviewer-facing documents and implementation choices. Testing is therefore not optional.

13.2 Test categories

13.2.1 Unit tests

- metadata parsing - path normalization - schema validation - duplicate detection - CLI command behavior

13.2.2 Integration tests

- DOI/arXiv/PDF ingestion end-to-end - PDF extraction on known sample papers - summary file creation - paper-to-code link resolution - claim-support audit generation

13.2.3 Platform tests

- Linux-native paths - WSL2 Linux paths - Windows path-handling assumptions where applicable

13.2.4 Failure-case tests

- malformed PDF - missing metadata - duplicate DOI - bad Unicode path - stale link to code file

13.2.5 Assistant adapter tests

- MCP tool schema correctness - CLI output consistency - workspace export file generation

13.3 Representative fixture set

The system should ship with a small fixture library of papers used for repeated testing:

- one clean arXiv paper,

- one DOI journal paper,
- one malformed PDF,
- one scanned/OCR-requiring PDF,
- one paper with many citations,
- one paper with ambiguous metadata.

13.4 Subsystem failure-mode appendix summary

The proposal should explicitly acknowledge that the main engineering risk is not algorithmic elegance but operational brittleness. The table below is a compressed failure-mode register; a full implementation should keep the same logic in a machine-readable issue or runbook format.

Subsystem	**Failure mode**	**Likely consequence**	**Mitigation class**
Paper ingestion	malformed or encrypted PDF	missing or corrupted extracted text	preserve raw PDF, mark extraction status, allow manual fallback
Metadata resolver	conflicting DOI/OpenAlex/arXiv records	wrong title/authors/year, duplicate papers	reconciliation rules + human review path
Deduplication	two records for one paper or one ID for two papers	broken links and synthesis errors	canonical ID rules + duplicate conflict resolution
Structured summary	overconfident generated summary	misleading literature synthesis	draft status, curation, claim-support audit
Paper-code links	stale file paths after refactor	false traceability	periodic stale-link checks
Query layer	stale or incomplete index	missing search results	explicit reindex and cache invalidation
MCP adapter	tool schema drift	assistant integration failure	adapter conformance tests
CLI adapter	path or shell incompatibility	non-portable workflows	shell-agnostic command design

Workspace exports	stale generated context files	assistants reading old context	export versioning and refresh commands
Shared store	unauthorized edits or accidental corruption	trust loss in research memory	append-only logs / backups / review workflow

13.5 Platform test matrix

A practical implementation should maintain a visible platform test matrix. A minimum starting matrix is:

Platform	**Execution mode**	**Critical checks**
Linux	native	ingestion, metadata resolution, CLI, MCP, file permissions
WSL2	Linux guest on Windows host	path normalization, file watching, DNS/network, mounted-drive behavior
Windows	optional direct support or constrained support statement	path quoting, PowerShell handling, workspace exports
Mixed IDE setup	VS Code / Cursor / Copilot clients	adapter invocation, generated context visibility, shared store access

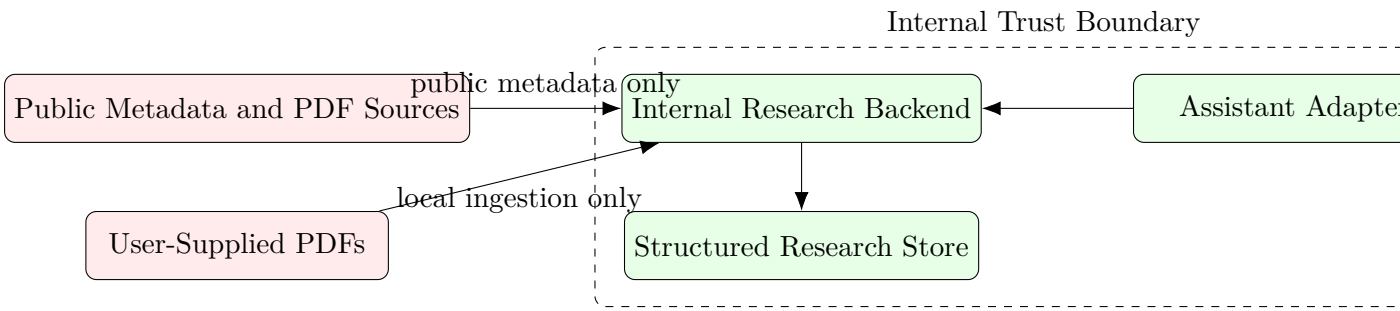
Chapter 14

Security, Privacy, and Governance

14.1 Security posture

The default stance should be local-first and least-privilege. Public paper metadata can be fetched externally, but internal notes, code links, and institution-specific project interpretations should remain under internal control.

14.2 Security and trust-boundary diagram



14.3 Governance requirements

The system should support:

- separation of public and internal content,
- explicit curation status,
- traceability of generated summaries,
- ownership of shared indices,
- and documented escalation paths for bugs and schema changes.

14.4 Security design rules

The first implementation should obey at least the following rules:

1. raw papers and extracted text remain local by default;
2. only public metadata queries go to external APIs unless explicitly approved otherwise;

3. internal code paths, notes, and review artifacts are never sent to public literature services;
4. assistant adapters are not allowed to invent persistent state outside the backend-defined store;
5. every generated summary must preserve provenance fields.

Chapter 15

Delivery Plan and Operational Ownership

15.1 Phase 1: local MVP

Deliverables:

- ingestion scripts,
- structured schema,
- paper store,
- metadata resolution,
- extracted text,
- summary drafts,
- CLI queries.

15.2 Personal desktop POC chapter

Before division-wide rollout, the most practical proof-of-concept is a fully functional personal version for Claude Code on one desktop, with optional use from a Cursor terminal on the same machine. This smaller scope is strategically useful because it reduces the hardest engineering variables while still demonstrating the core value of the system.

15.2.1 Why the personal POC is the right first build

The personal POC removes several difficult classes of problems:

- no multi-user consistency or permissions coordination,
- no central shared service requirement,
- no mandatory Copilot parity in the first release,
- no organization-wide schema governance before the workflow is proven,
- and much simpler operational debugging.

At the same time, the POC should remain *functionally complete*, so that it demonstrates the real workflow rather than a toy subset.

15.2.2 Functional scope of the personal POC

The personal POC should support all of the following on a single machine:

1. ingest paper from PDF / DOI / arXiv / URL,
2. extract text and resolve metadata,
3. create a structured paper summary,
4. create and query paper-to-code and paper-to-document links,
5. expose retrieval through CLI,
6. expose retrieval through Claude Code via MCP,
7. allow Cursor-terminal usage through the same CLI,
8. generate claim-support audits,
9. generate reviewer-facing mini syntheses.

This scope is deliberately narrow enough to be buildable, but broad enough to prove the actual concept.

15.2.3 POC-specific file layout

A personal version can simplify the larger architecture by co-locating the research store with the user's working environment:

```
research-assistant-poc/
  README.md
  pyproject.toml
  src/
    research_assistant/
      ingest/
      schemas/
      storage/
      summarize/
      links/
      query/
      adapters/
      tests/
  local_research/
    papers/
      raw/
      extracted/
    metadata/
    summaries/
    links/
    reviews/
    indices/
  examples/
  docs/
```

15.2.4 POC command surface

The personal version should feel complete from day one. A minimum useful command surface is:

```
ra ingest --pdf paper.pdf
ra ingest --doi 10.xxxx/xxxxx
ra ingest --arxiv 1903.03704
ra find --query "transport maps hmc"
ra show --paper-id paper_hoffman_2021_neutra
ra citing --paper-id paper_hoffman_2021_neutra
ra link add --paper-id p1 --target docs/monograph.tex --relationship supports
ra link add --paper-id p1 --target src/model.py --relationship implements
ra audit-claim --claim-file claim.txt --papers p1 p2
ra synthesize --topic "transport maps for posterior geometry"
```

15.2.5 POC MCP surface for Claude Code

The Claude Code integration should initially expose only the most useful operations:

- `find_paper`
- `get_paper_summary`
- `papers_citing`
- `paper_code_links`
- `claim_support_audit`
- `reviewer_synthesis`

This is enough to make the assistant meaningfully research-aware without making the first implementation too broad.

15.2.6 POC success criteria

A personal POC should be judged successful if it can support the following end-to-end demo:

1. ingest a real research paper,
2. create a usable structured summary,
3. link the paper to a code file and a monograph section,
4. answer a Claude Code query about the paper and its implementation relevance,
5. audit whether a paragraph is supported by citations,
6. generate a reviewer-facing technical note.

If that demo works well on a single machine, the case for group expansion is substantially stronger.

15.2.7 Why this POC remains aligned with the larger architecture

The personal POC is not a throwaway prototype. It should be built with the same schemas, module boundaries, and retrieval interfaces planned for the larger system. The difference is only deployment scope and operational complexity.

15.2.8 Suggested staffing model

A realistic initial staffing model is:

- **one primary engineer-owner** responsible for architecture, code integration, and release discipline;
- **one or two research-domain reviewers** who validate paper summaries, claim-support rules, and field-specific usefulness;
- **optional part-time platform advisor** for infrastructure or security review if group policy requires it.

This staffing model is designed to be achievable in a research division. It does not assume a dedicated full-scale platform team.

15.2.9 Operational ownership model

The following ownership categories should be defined explicitly from the start:

Area	Recommended owner
Core backend code	primary engineer-owner
Schema changes	primary engineer-owner + research reviewer sign-off
Paper curation conventions	research reviewers
Dependency updates	primary engineer-owner
Assistant adapter compatibility	primary engineer-owner, with user feedback from each tool community
Support and bug triage	shared queue, but one named owner

15.2.10 MVP sprint-by-sprint execution plan

A practical MVP can be broken into short implementation sprints.

Sprint 1: core paper store and ingestion. Implement:

- file layout,
- paper ID assignment,
- raw PDF storage,
- metadata resolution,
- text extraction,
- draft summary file creation.

Sprint 2: retrieval and CLI. Implement:

- paper lookup,
- summary retrieval,
- citation graph lookup,
- link creation,
- CLI commands.

Sprint 3: audit and review workflows. Implement:

- claim-support audit,
- reviewer synthesis generation,
- curation-status transitions,
- stale-link check.

Sprint 4: Claude Code adapter and workspace exports. Implement:

- MCP tools,
- workspace export files,
- integration tests.

Sprint 5: cross-tool validation. Validate:

- Cursor workflow,
- Copilot workflow via CLI/workspace files,
- WSL2/Linux path handling,
- user onboarding instructions.

15.3 Phase 2: assistant integration

Deliverables:

- MCP server,
- CLI adapter for all environments,
- workspace export files,
- minimal shared conventions.

15.4 Phase 3: review and audit workflows

Deliverables:

- claim-support audit engine,
- paper-to-code traceability workflows,
- reviewer synthesis generation,
- stale-link checks.

15.5 Phase 4: group scaling

Deliverables:

- shared repository templates,
- optional central index,
- onboarding documentation,
- optional web UI.

15.6 Ownership model

At minimum, the system needs explicit ownership for:

- schema changes,
- dependency updates,
- ingestion failures,
- integration bugs,
- and onboarding/support.

Chapter 16

Anticipated Skeptical Questions and Preemptive Answers

16.1 Why not just use SciSpace?

Because SciSpace is paper-aware but not code-aware. It does not know our repositories, implementation files, tests, or reviewer-facing technical traceability obligations.

16.2 Why not just use Claude Code or Cursor better?

Because the missing layer is not only better prompting. The missing layer is durable, structured, shared research memory.

16.3 Why should this be cross-platform?

Because the division already uses multiple assistants, and a single-assistant design would reduce adoption and create lock-in.

16.4 Can this team realistically build it?

Yes, if the first version is deliberately simple:

- file-first,
- schema-driven,
- backend-plus-adapters,
- local-first,
- phased delivery,
- narrow MVP.

The proposal is intentionally designed to reduce platform complexity early, not increase it.

16.5 Why should management believe this is not vaporware?

Because the proposal deliberately exposes implementation details, platform risks, module boundaries, file layout, testing strategy, and phased delivery constraints. It is designed to be close to implementation, not merely visionary.

16.6 How Claude Code could implement from this document

This document is intentionally written at a level where Claude Code or a human engineer could begin implementation directly from it. Specifically, the document now contains:

- a repository/module tree,
- file-by-file responsibilities,
- formal record constraints,
- MCP and CLI contracts,
- ingestion pseudocode,
- metadata reconciliation and deduplication pseudocode,
- subsystem failure modes,
- platform test matrix,
- and a sprint-by-sprint MVP sequence.

The intended usage is not blind code generation from prose. Rather, the proposal is sufficiently structured that a coding assistant can implement one subsystem at a time without inventing the architecture on the fly.

16.6.1 Suggested implementation appendix workflow

A practical execution pattern using Claude Code would be:

1. implement schemas and file-store primitives first;
2. implement ingestion and metadata resolution second;
3. add CLI retrieval commands third;
4. add MCP adapters only after the backend is testable;
5. validate each subsystem with fixture papers before enabling broader team use.

This ordering matters. It prevents assistant integrations from being built on top of unstable backend assumptions.

Chapter 17

Decision Memo for Management Review

17.1 Decision requested

Approve a phased build of a shared, assistant-agnostic research-development backend for the division.

17.2 What approval enables

Approval would authorize:

1. MVP implementation of the local-first backend;
2. pilot integration with Claude Code and one additional assistant workflow;
3. schema and retrieval layer validation on real research projects;
4. preparation for broader team rollout if pilot value is demonstrated.

17.3 What approval does not yet commit the division to

Approval of this proposal does *not* commit the division to:

- building a full enterprise platform immediately,
- mandatory adoption by all teams,
- a web product or central service in phase 1,
- replacing existing assistants or paper tools.

17.4 Recommended decision

Approve the MVP phase with the explicit understanding that:

- the design is backend-first and cross-platform by construction,
- the MVP is deliberately narrow and auditable,
- platform and environment risks are recognized in advance,
- and continuation beyond the MVP should depend on demonstrated workflow value.